CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: __Preston Gaskill_____

____N/A_____

_____

Project Teams Group #:_____N/A_____

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

IF/ID

1. s_Inst
2. currentInst

ID/EX

3. Current Instruction
4. Branch_Wire
5. Wire_ALUSrc
6. ALUControl
7. ImmType
8. ltSrc
9. MemWrite
10. PCSrc
11. RegDst
12. Halt
13. WritePCORResultSrc
14. ALUOutORPC
15. PCAddress
16. Regwrite
17. ZeroMuxWrite
18. Zero
19. Data 1
20. Data 2
21. FetchLogicAddress

EX/MEM

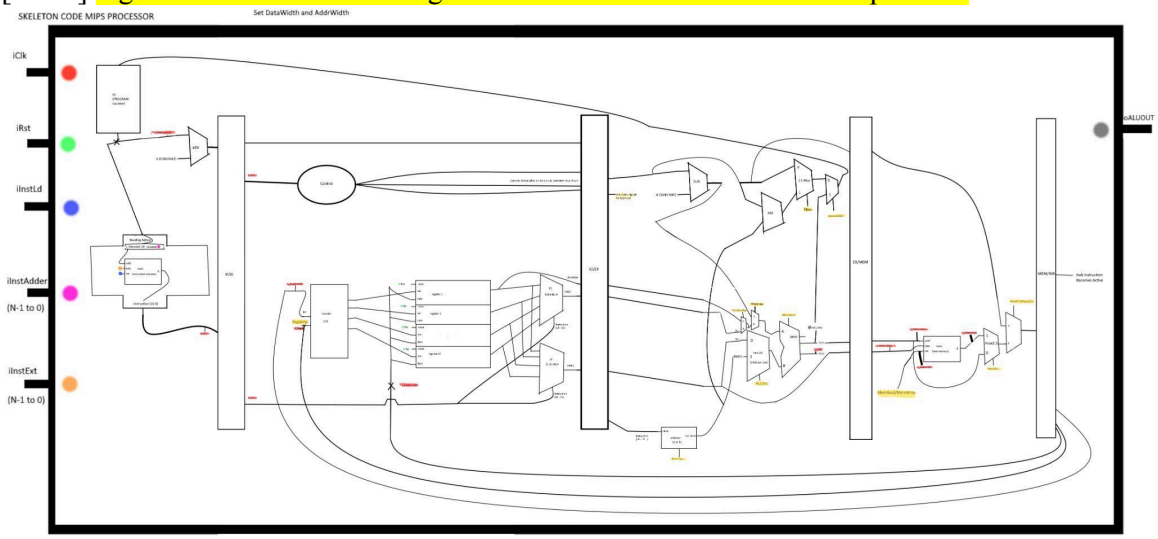
22. pcPlus4
23. ALUOut

24. Data 2
25. Opcode
26. Funct7
27. Rd
28. Regwrite
29. Memwrite
30. Resultsrc
31. writePCORResultSrc

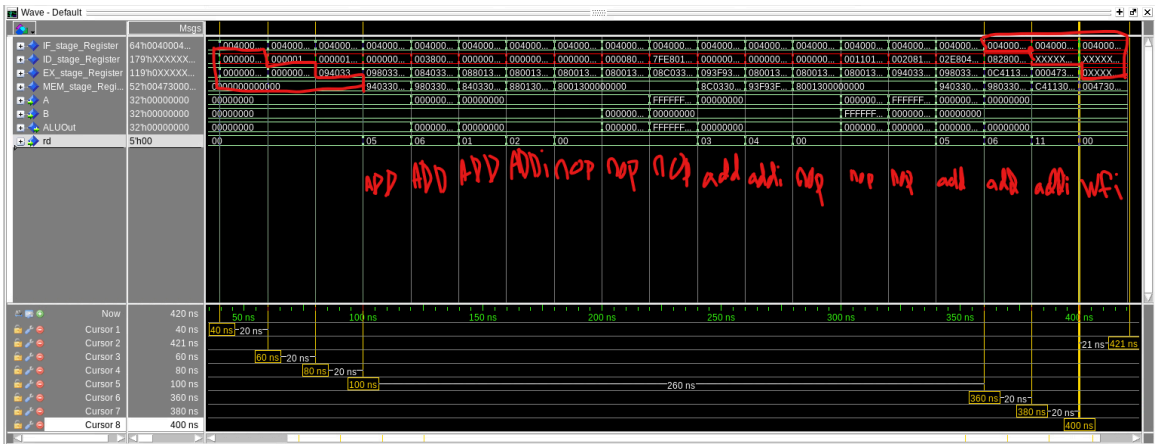
MEM/WB

32. Final data value for register
33. Opcode
34. Funct6
35. Rd location
36. Write enable signal

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.



In the above diagram, you can see the ‘step’ looking output from the register stages and how they grow until the processor runs out of instructions (note that the control logic initially outputs a value at the start of the IF stage, but this doesn’t matter), and the ‘step’ pattern repeats in the opposite direction.

Our first write-back occurs after the 100ns cursor, and this continues until we reach our wfi instruction and the halt value is set (although it is not visible in the waveform, the change is instantaneous).

```
add x5, x0, x0
add x6, x0, x0
add x1, x0, x0

addi x2, x0, 7
nop
nop
nop
add x3, x2, x0

addi x4, x0, -3
nop
nop
nop
add x5, x4, x2

add x6, x2, x4

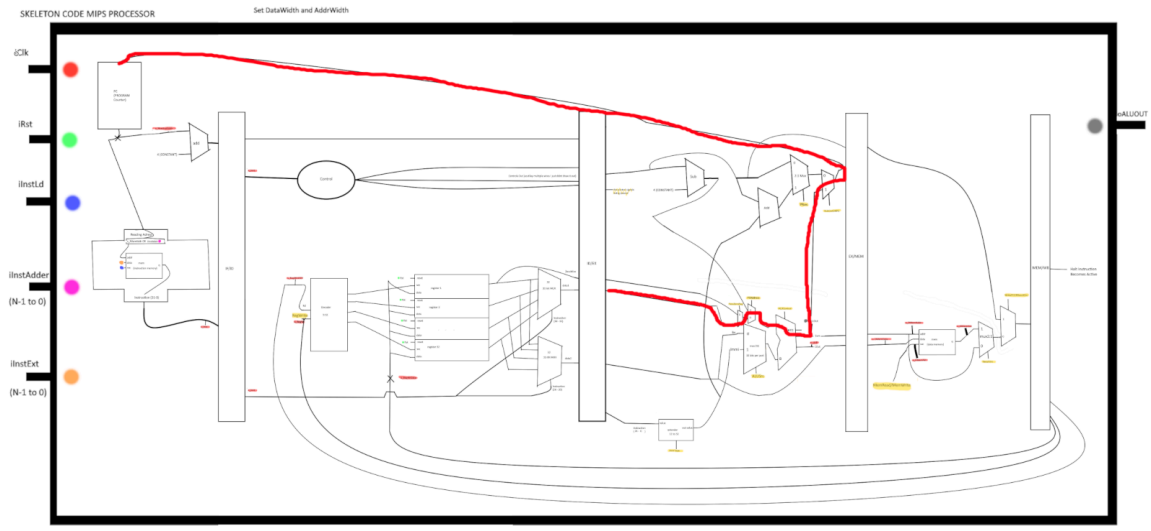
addi a7, x0, 93
wfi
```

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

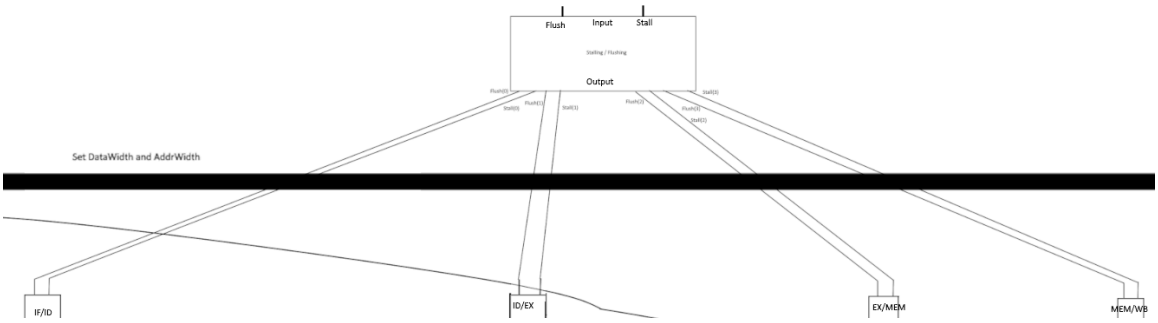
OMIT

[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

maximum frequency: 36.44mhz



[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.

tb_PipelineregisterUpdate.vhd



To start, stall nothing, so you can see how the data (0x00000004) is loaded into each register (1-4).

And this continues when a new data value (0x00000002) is then loaded into each register.

And then we try to load in a new value (0x00000003) into all the registers again, but register 2 and register 4 are stalled, so the value is loaded only into register 1, and none of the other registers change their values.

Finally, register 3 gets flushed, and you can see its value get overwritten with 0x00000000.

The pattern continues normally, with only the first register being updated because it is the only one that is not stalled and receiving new values.

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

I did not do controls

add
IF/DE: IF_stage_Register
DE/EX: ID_stage_Register
EX/MEM: EX_stage_Register
MEM/WB: MEM_stage_Register

addi
IF/DE: IF_stage_Register
DE/EX: ID_stage_Register
EX/MEM: EX_stage_Register
MEM/WB: MEM_stage_Register

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

add
IF/DE: S_Inst

DE/EX: register1TOALU, register2TOALU
EX/MEM: s_ALUOut
MEM/WB: wireTOWB

addi
IF/DE: S_Inst
DE/EX: register1TOALU, S_Inst
EX/MEM: s_ALUOut
MEM/WB: wireTOWB

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

OMIT, no forwards so we are only stalling

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF Stage

```
i_D(31 downto 0) => s_Inst, --Cur
i_D(63 downto 32) => currentInst,
```

ID Stage

```
i_D(31 downto 0) => IF_stage_Register(63 downto 32),
i_D(32) => Branch_Wire, --Branch_Wire
i_D(33) => Wire_ALUSrc, --Wire_ALUSrc
i_D(37 downto 34) => Wire_ALUControl, --Wire_ALUContr
i_D(40 downto 38) => Wire_ImmType, --Wire_ImmType
i_D(41) => Wire_ResultSrc, --Wire_ResultSrc
i_D(42) => Wire_MemWrite, --Wire_MemWrite
i_D(43) => Wire_PCSrc, --Wire_PCSrc
i_D(44) => Wire_RegDst, --Wire_RegDst
i_D(45) => halt_Wire, --halt_Wire
i_D(46) => s_WritePCORResultSrc, --s_WritePCORResultSr
i_D(47) => s_ALUOutORPC, --s_ALUOutORPC
i_D(48) => s_PCAddress, --s_PCAddress
i_D(49) => Wire_RegWrite, --Wire_RegWrite
i_D(50) => s_zeroMuxWire, --s_zeroMuxWire
i_D(82 downto 51) => doINeedAWireHere, --zer
i_D(114 downto 83) => register1TOALU, --Data 1
i_D(146 downto 115) => register2TOALU, --Data 2
i_D(178 downto 147) => IF_stage_Register(31 downto 0)
```

EX Stage

```
i_D(31 downto 0) => s_pcPLUS4, --s_pcPLUS4
i_D(63 downto 32) => s_ALUOut, --ALu out
i_D(95 downto 64) => ID_stage_Register(146 downto 115),
i_D(102 downto 96) => ID_stage_Register(153 downto 147),
i_D(109 downto 103) => ID_stage_Register(178 downto 172)
i_D(114 downto 110) => ID_stage_Register(158 downto 154)
i_D(115) => ID_stage_Register(49), --Reg Write
i_D(116) => ID_stage_Register(42), --Mem Write
i_D(117) => ID_stage_Register(41), --ResultSRC
i_D(118) => ID_stage_Register(46), --WritePCORResultSRC
```

MEM Stage

```
i_D(31 downto 0) => wireTOWB, --Final data value for r
i_D(38 downto 32) => EX_stage_Register(102 downto 96),
i_D(45 downto 39) => EX_stage_Register(109 downto 103)
i_D(50 downto 46) => EX_stage_Register(114 downto 110)
i_D(51) => EX_stage_Register(115), --write enable sign
```

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

BRANCH INSTRUCTIONS
JAL
JALR

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

BRANCH INSTRUCTIONS

- For a branch at any point, flush the ID and EX register in the following clock cycle.

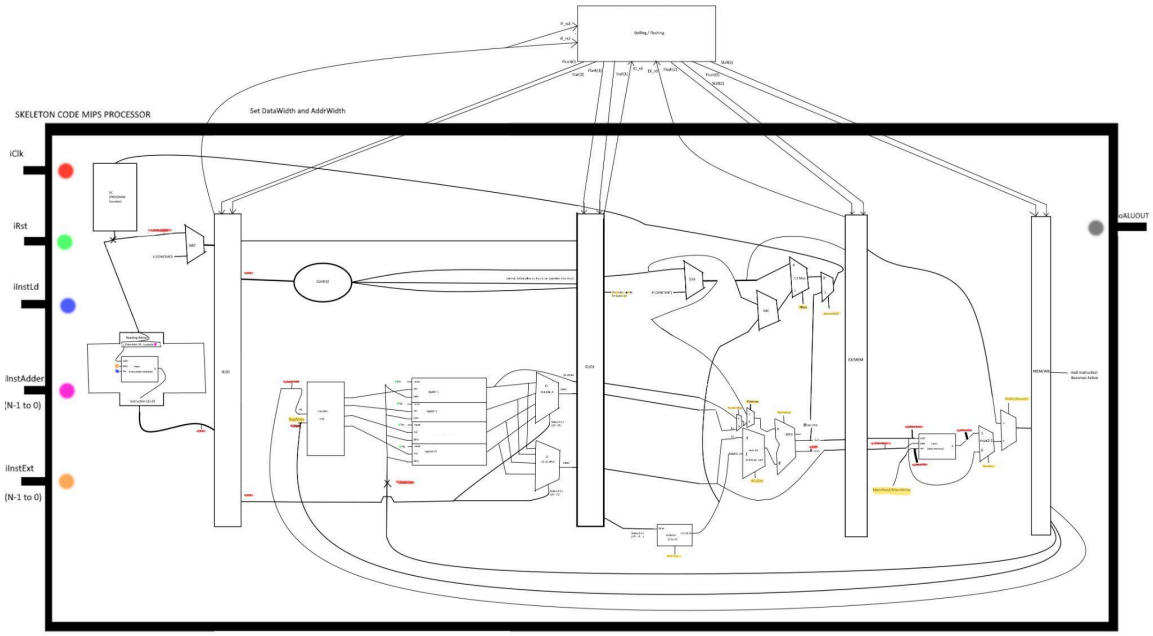
JAL

- For a jal, we flush the IF and ID registers on the second cycle and stall the IF register with that bubble value.

JALR

- For jalr, we flush the second register and then stall the first register in the following clock cycle.

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.

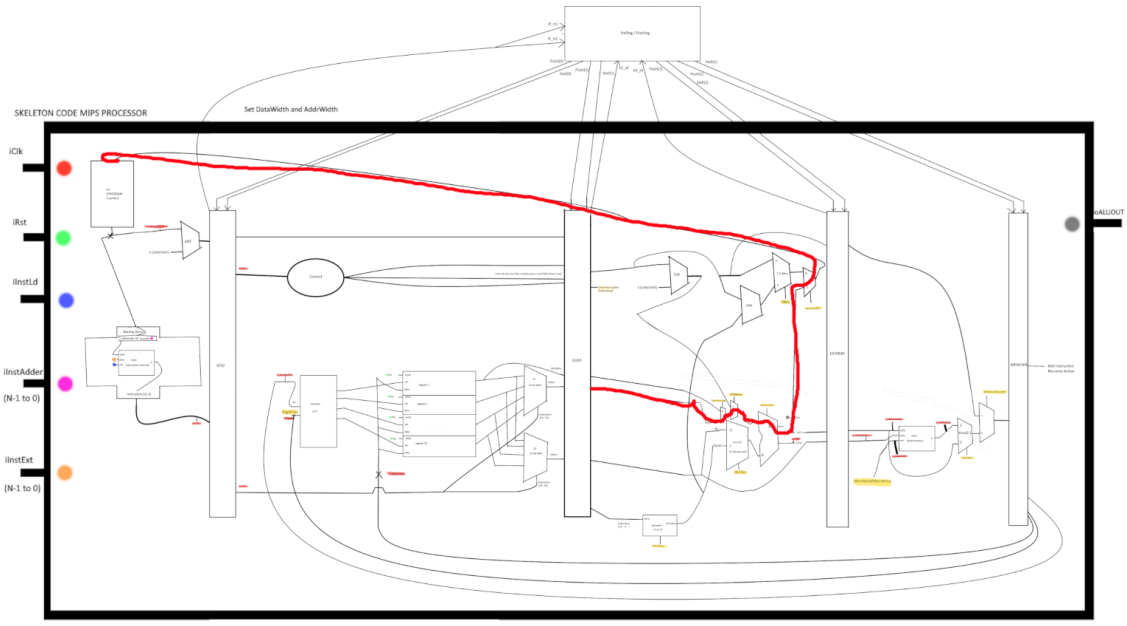


[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).



Maximum Frequency: 44.05 MHz